Get unlimited access to the best of Medium for less than $1/week. Become a member

OpenTelemetry on Kubernetes

Deploying an OpenTelemetry Collector on Kubernetes

Magsther Follow 7 min read · Aug 11, 2022

148 2

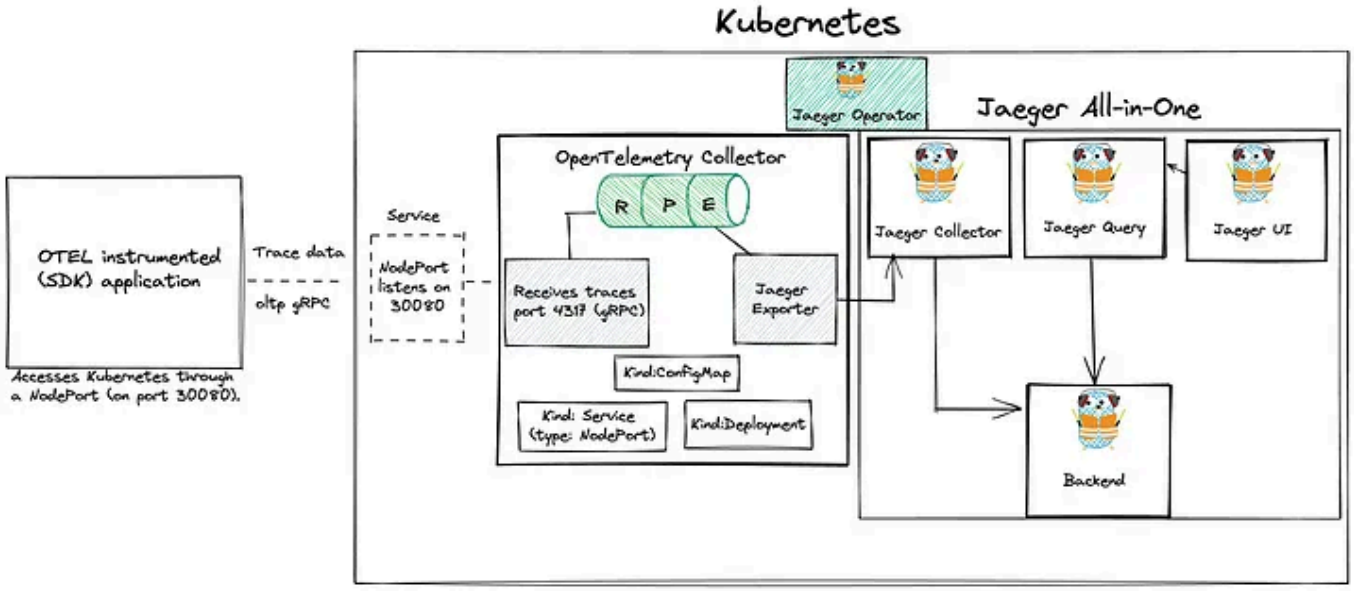
In the A beginner's guide to OpenTelemetry post , we looked at what you need to know to get started with OpenTelemetry.

Today, we will deploy an **OpenTelemetry** collector on **Kubernetes.**

Once it is deployed, we use an OTEL instrumented (Go) **application** provided by OpenTelemetry to send traces to the **Collector.**

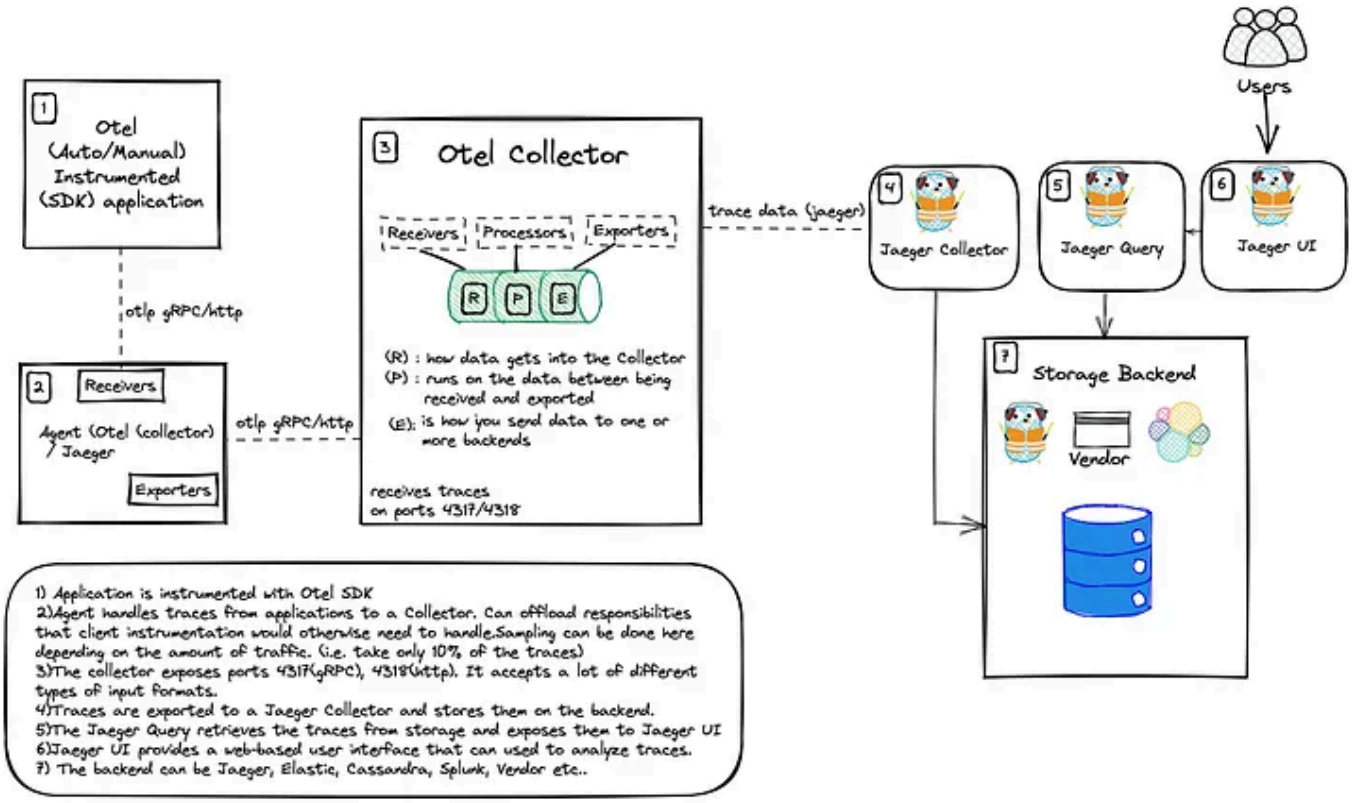
From there, we will bring the trace data to a **Jaeger collector.** Finally, the traces will be visualized using the **Jaeger UI.**

The image below shows the flow between the **application** — OpenTelemetry **collector** and **Jaeger.**



> *This OpenTelemetry repository provides a complete demo on how you can deploy OpenTelemetry on Kubernetes,* ***we can use this as a starting point.***

If you recall from A beginner's guide to OpenTelemetry our **OTEL** setup looks like this:



OpenTelemetry solution

To start off, we need a **Kubernetes** cluster. In our case, we will use a local instance of Kind.

The application will access this Kubernetes cluster through a NodePort (on port 30080).

```
conn, err := grpc.DialContext(ctx, "localhost:30080",
grpc.WithTransportCredentials(insecure.NewCredentials()),
grpc.WithBlock())
```

To use **NodePort** with **Kind**, we need to first enable it.

> Extra port mappings can be used to port forward to the kind nodes. This is a cross-platform option to get traffic into your kind cluster.

```
vim kind-config.yaml
```

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane
  # port forward 30080 on the host to 30080 on this node
  extraPortMappings:
  - containerPort: 30080
    hostPort: 30080
- role: worker
```

Create the cluster with: `kind create cluster --config kind-config.yaml`

```
Creating cluster "kind" ...
 ✓ Ensuring node image (kindest/node:v1.24.0) 🖼
 ✓ Preparing nodes 📦 📦
 ✓ Writing configuration 📜
 ✓ Starting control-plane 🕹️
 ✓ Installing CNI 🔌
 ✓ Installing StorageClass 💾
 ✓ Joining worker nodes 🚜
Set kubectl context to "kind-kind"
You can now use your cluster with:

kubectl cluster-info --context kind-kind

Thanks for using kind! 😊
```

Once our Kubernetes cluster is up, we can start with deploying **Jaeger**.

What is Jaeger?

Jaeger is open source distributed tracing system for tracing transactions between distributed services. It's used for monitoring and troubleshooting complex microservices environments. By doing this, we can view traces and analyses the application's behavior.

Why do we need it?

Using a tracing system (like Jaeger) is especially important in microservices environment, since they are considered lot more difficult to debug than a single **monolithic** application.

What problems does Jaeger solve?

- Distributed tracing monitoring
- Performance and latency optimization
- Root cause analysis
- Service dependency analysis

Deploy Jaeger

To deploy Jaeger on Kubernetes cluster, we can make use of the Jaeger operator.

> *Operators are pieces of software that ease the operational complexity of running another piece of software.*

Jaeger Operator

You first need to install the **Jaeger Operator** on Kubernetes. This operator will then watch for new Jaeger custom resources (CR).

There are different ways of installing the **Jaeger Operator** on Kubernetes:

- using **Helm**
- using **Deployment** files

> *Before you start, pay attention to the Prerequisite section.*

Since version 1.31 the Jaeger Operator uses webhooks to validate Jaeger custom resources (CRs). This requires an installed version of the **cert-manager.**

Installing Cert-Manager

cert-manager is a powerful and extensible X.509 certificate controller for Kubernetes and OpenShift workloads. It will obtain certificates from a variety of Issuers, both popular public Issuers as well as private Issuers, and ensure the certificates are valid and up-to-date, and will attempt to renew certificates at a configured time before expiry.

Installation of **cert-manager** of is very simple, just run:

```
kubectl apply -f https://github.com/cert-manager/cert-manager/releases/download/v1.9.1/cert-manager.yaml
```

> *By default, **cert-manager** will be installed into the cert-manager namespace.*

You can verify the installation by following the instructions here

With **cert-manager** installed, let's continue with the deployment of **Jaeger**

Installing Jaeger Operator using Helm

Jump over to Artifact Hub and search for jaeger-operator

Add the Jaeger Tracing Helm repository:

```
helm repo add jaegertracing https://jaegertracing.github.io/helm-charts
```

To install the chart with the release name `my-release` (in the default namespace)

```
helm install my-release jaegertracing/jaeger-operator
```

You can also install a specific version of the helm chart:

```
helm install my-release jaegertracing/jaeger-operator --version
2.25.0
```

Verify that it's installed on Kubernetes:

```
helm list -A
```

> *You can also deploy the Jaeger operator using **deployment files**.*
>
> ```
> kubectl create -f https://github.com/jaegertracing/jaeger-operator/releases/download/v1.36.0/jaeger-operator.yaml
> ```

At this point, there should be a **jaeger-operator** deployment available.

```
kubectl get deployment my-jaeger-operator
```

```
NAME                 READY   UP-TO-DATE   AVAILABLE   AGE
my-jaeger-operator   1/1     1            1           2m58s
```

The operator is now ready to create Jaeger instances.

Deploy Jaeger All-in-One

The **operator** that we just installed doesn't do anything itself, it just means that we can create **jaeger** resources/instances that we want the jaeger operator to manage.

The simplest possible way to create a Jaeger instance is by deploying the **All-in-one strategy**, which installs the `all-in-one` image, that includes the **agents**, **collector**, **query** and the **Jaeger UI** in a single pod using in-memory storage.

Create a yaml file like the following. The name of the Jaeger instance will be `simplest`

```
vim simplest.yaml
```

```
apiVersion: jaegertracing.io/v1
kind: Jaeger
metadata:
  name: simplest
```

```
kubectl apply -f simplest.yaml
```

After a little while, a new in-memory all-in-one instance of Jaeger will be available, suitable for quick demos and development purposes.

When the Jaeger instance is up and running, we can check the **pods** and **services**.

```
kubectl get pods
```

```
NAME                      READY STATUS    RESTARTS   AGE
simplest-656d7cf5c8-lff7b 1/1  Running   0          3m55s
```

```
kubectl get services
```

To get the pod name, query for the pods belonging to the simplest Jaeger instance:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	18m
my-jaeger-operator-metrics	ClusterIP	10.96.129.254	<none>	8383/TCP,8686/TCP	9m54s
simplest-agent	ClusterIP	None	<none>	5775/UDP,5778/TCP,6831/UDP,6832/UDP	5m31s
simplest-collector	ClusterIP	10.96.210.184	<none>	9411/TCP,14250/TCP,14267/TCP,14268/TCP	5m31s
simplest-collector-headless	ClusterIP	None	<none>	9411/TCP,14250/TCP,14267/TCP,14268/TCP	5m31s
simplest-query	ClusterIP	10.96.19.215	<none>	16686/TCP	5m31s

Query the logs from the pod:

```
kubectl logs -l app.kubernetes.io/instance=simplest
```

```
{"level":"info","ts":1660155049.86027,"caller":"channelz/logging.go:5
0","msg":"[core]Channel Connectivity change to
READY","system":"grpc","grpc_log":true}
{"level":"info","ts":1660155049.8612773,"caller":"grpc/builder.go:120
","msg":"Agent collector connection state
change","dialTarget":":14250","status":"READY"}
{"level":"info","ts":1660155049.8617437,"caller":"app/server.go:241",
"msg":"Starting HTTP server","port":16686,"addr":":16686"}
{"level":"info","ts":1660155049.8621716,"caller":"app/server.go:260",
"msg":"Starting GRPC server","port":16685,"addr":":16685"}
```

Jaeger UI

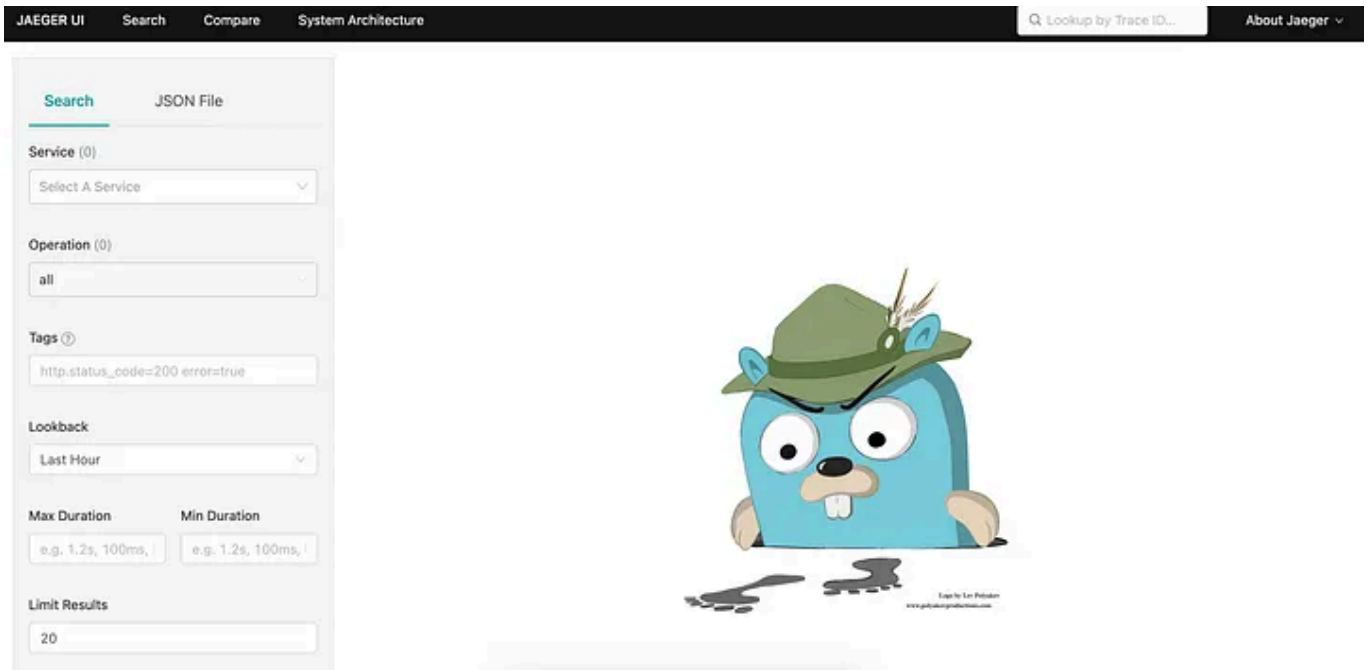
To access application in your Kubernetes cluster, we can use port forwarding.

To access the Jaeger UI, run:

```
kubectl port-forward svc/simplest-query 16686:16686
```

```
Forwarding from 127.0.0.1:16686 -> 16686
Forwarding from [::1]:16686 -> 16686
```

Now open a browser to access the Jaeger UI



Jaeger UI

Open in app ↗

Medium Search Write

Deploy the OpenTelemetry Collector

To deploy the OpenTelemetry collector, we will use this otel-collector.yaml file as a starting point. The yaml file consists of a **ConfigMap**, **Service** and a **Deployment**.

```
vim otel-collector.yaml
```

Make sure to change the name of the **jaeger collector** (exporter) to match the one we deployed above. In our case , that would be:

```
exporters:
      jaeger:
        endpoint: "simplest-
collector.default.svc.cluster.local:14250"
```

Also, pay attention to **receivers**. This part creates the receiver on the Collector side, and open up port `4317` for receiving traces, which enables the application to send data to the OpenTelemetry Collector.

```
...
  otel-collector-config: |
    receivers:
      otlp:
        protocols:
          grpc:
            endpoint: "0.0.0.0:4317"
...
```

Apply the file with: `kubectl apply -f otel-collector.yaml`

```
configmap/otel-collector-conf created
service/otel-collector created
deployment.apps/otel-collector created
```

Verify that the the OpenTelemetry Collector is up and running.

```
kubectl get deployment
```

```
kubectl logs deployment/otel-collector
```

```
"Everything is ready. Begin running and processing data."
```

Run the application

Time to send some trace data to our OpenTelemetry collector.

> Remember, that the application access the Kubernetes cluster through a NodePort on port 30080. The Kubernetes service will bind the `4317` port used to access the OTLP receiver to port `30080` on the Kubernetes node.
>
> By doing so, it makes it possible for us to access the Collector by using the static address `<node-ip>:30080`. In case you are running a local cluster, this will be `localhost:30080`. Source

This repository contains an (SDK) instrumented application written in **Go**, that simulates an application.

```
go run main.go
```

```
2022/08/10 20:31:37 Waiting for connection...
2022/08/10 20:31:37 Doing really hard work (1 / 10)
2022/08/10 20:31:38 Doing really hard work (2 / 10)
2022/08/10 20:31:39 Doing really hard work (3 / 10)
2022/08/10 20:31:40 Doing really hard work (4 / 10)
2022/08/10 20:31:41 Doing really hard work (5 / 10)
2022/08/10 20:31:42 Doing really hard work (6 / 10)
2022/08/10 20:31:43 Doing really hard work (7 / 10)
```

```
2022/08/10 20:31:44 Doing really hard work (8 / 10)
2022/08/10 20:31:45 Doing really hard work (9 / 10)
2022/08/10 20:31:46 Doing really hard work (10 / 10)
2022/08/10 20:31:47 Done!
```

Viewing the data

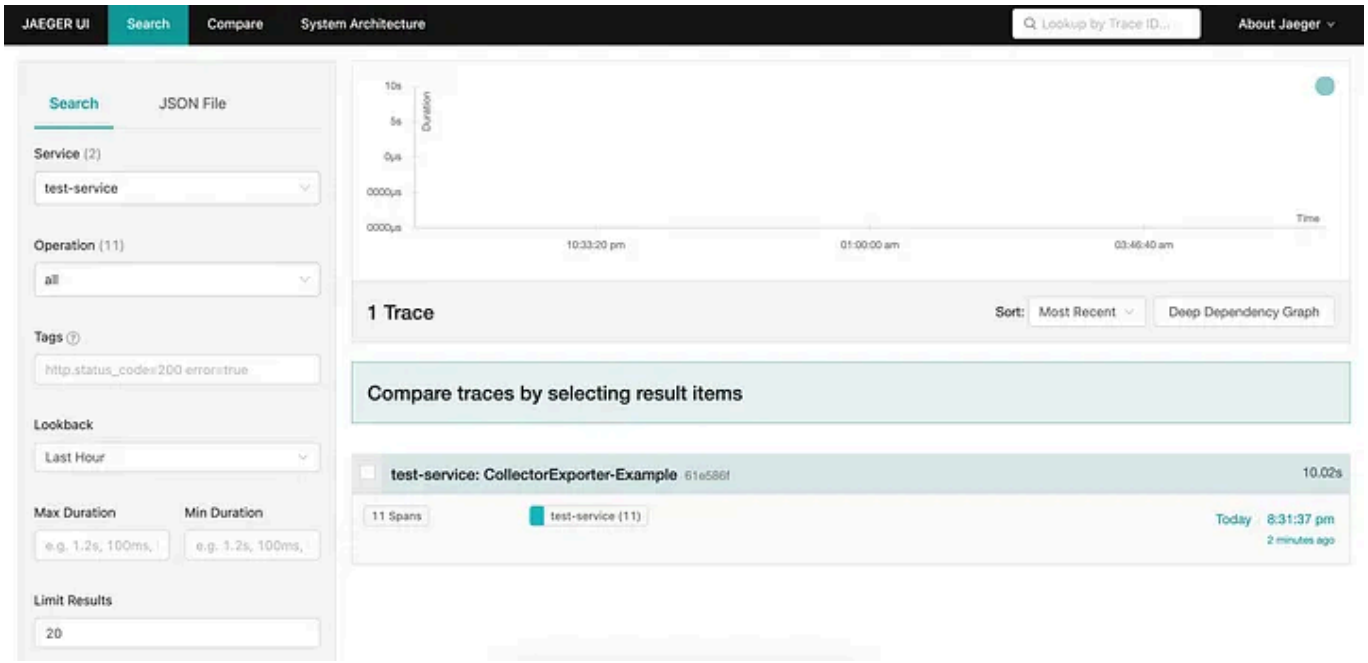
Let's check out the telemetry data generated by our sample application

Again, we can use port-forwarding to access Jaeger UI.

```
kubectl port-forward svc/simplest-query 16686:16686
```

Open web-browser and go to http://127.0.0.1:16686/

Under Service select **test-service** to view the generated traces.



The **service name** is specified in the `main.go` file.

```
res, err := resource.New(ctx,  resource.WithAttributes(   // the
service name used to display traces in backends
semconv.ServiceNameKey.String("test-service"),  ),
```

The application will access this Kubernetes cluster through a NodePort (on port 30080). The **URL** is specified here:

```
conn, err := grpc.DialContext(ctx, "localhost:30080",
grpc.WithTransportCredentials(insecure.NewCredentials()),
grpc.WithBlock()) if err != nil {  return nil, fmt.Errorf("failed to
create gRPC connection to collector: %w", err) }
```

That’s it

Conclusion

In this post we deployed an **OpenTelemetry** collector on **Kubernetes.** Then we sent trace data to this collector using a Otel **SDK** instrumented **application** written in Go. From there, the traces were sent to a Jaeger collector and visualised in **Jaeger UI.**

Opentelemetry Jaeger Development DevOps Kubernetes

Written by Magsther

789 followers · 4 following

Creator of “Awesome OpenTelemetry”. https://twitter.com/magsther

Follow

Responses (2)

Anand Ray

What are your thoughts?

Abdelfattah Sekak
Oct 3, 2023

Engaging writing, makes learning fun

1 Reply

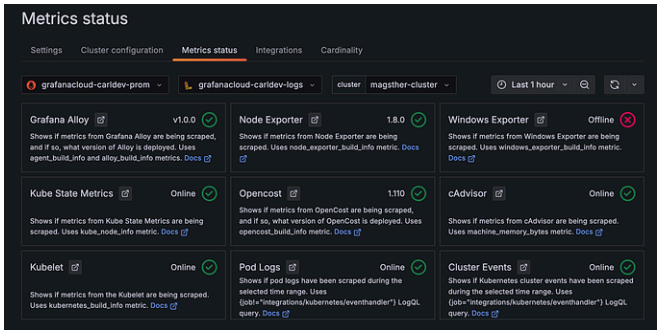
Mutha Nagavamsi
Oct 2, 2023

Thank you soo much for posting this.

Does it also work with latest version of kind kubernetes cluster?

Reply

More from Magsther

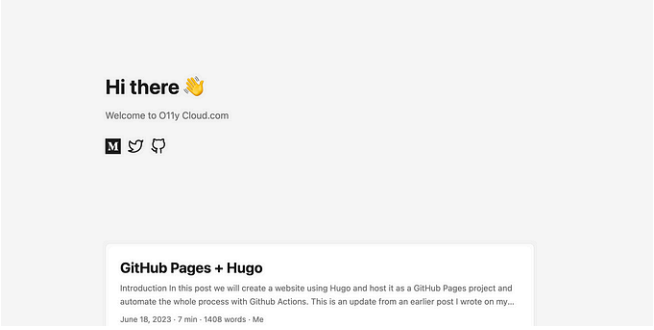


Magsther

Grafana Alloy & OpenTelemetry

Say Hello to Alloy

May 18, 2024 131 3

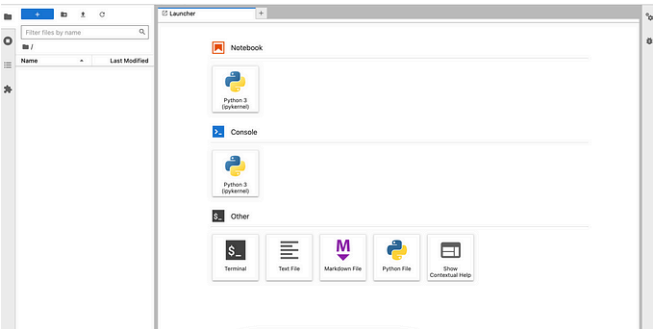


Magsther

How to Create and Host a Website with Hugo and GitHub Pages

GitHub Pages + Hugo

Jun 18, 2023 206 3

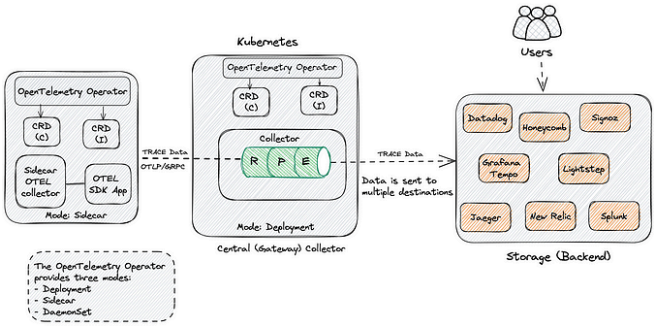


Magsther

JupyterHub on Kubernetes

An introduction to JupyterHub

Dec 29, 2022 29



Magsther

OpenTelemetry Up and Running

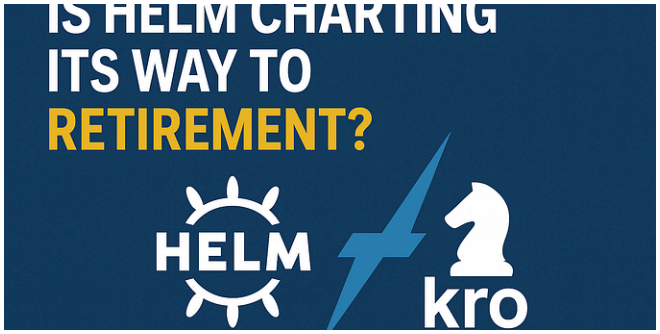
What you need to know about OpenTelemetry

Mar 28, 2023 123 1

See all from Magsther

Recommended from Medium

Yuri Fenyuk



Devlink Tips

Kubernetes observability with OpenTelemetry and Jaeger

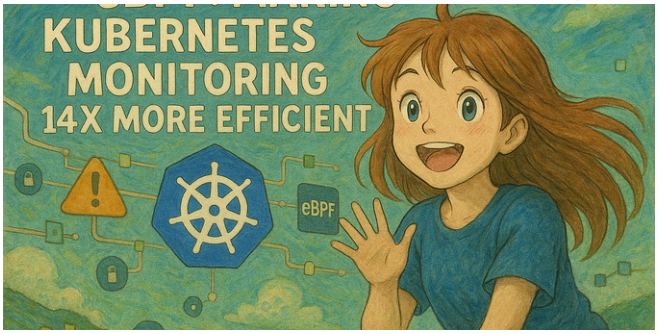
Having a Kubernetes cluster means hosting many computing resources inside. This...

Dec 10, 2024 6 1

Is Helm charting its way to retirement?

Kro enters the arena and challenges helm's legacy in kubernetes land

May 2 148 4

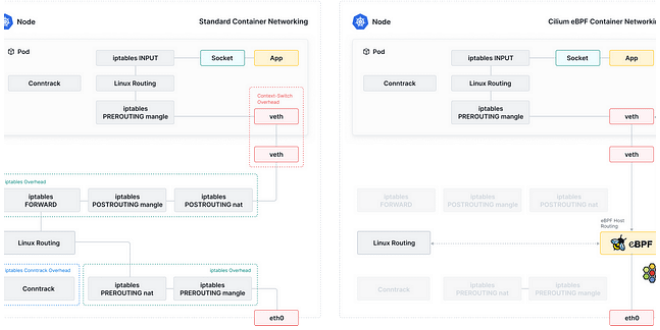


In devdotcom by Baivab Mukhopadhyay

eBPF: Making Kubernetes Monitoring 14X More Efficient

Ever got a 2 a.m. alert that your Kubernetes app is down? You're digging through logs,...

May 5 83 2



Quân Huỳnh

Kubernetes Networking: eBPF in Kubernetes—Replacing the Old...

Welcome back to eBPF in Action! In Part 3, we saw how eBPF works.

Apr 12 41

howtouselinux

PostgreSQL 18 Release Highlights: 7 New Features You Need to Know

The PostgreSQL community is back with something exciting—the first beta release of...

In Towards AWS by Mohamed ElEmam

The Critical Kubernetes Design Patterns Every Senior Engineer...

Unlock resilient, production-grade K8s architectures with patterns the pros won't tel...

6d ago 233 4 May 5 10

See more recommendations